TASK

Long-Running Job Task Description
* Please consider that AI or other tools are not allowed to solve the test.

General Idea
Simulate a hard processing task for input data items provided by the user.

Part 1

1. It should be the SPA with .NET 8 (or newer) on the backend and ReactJS (preferable)/Angular on the frontend, and this app should implement the following items below.
2. User can enter text into a text field.
3. User can press the “Process” button to get the following output string:
    - All unique characters of the input string, sorted, with the number of their occurrences next to each.
    - A slash (/).
    - The input string encoded in Base64 format.

- Processing should be performed on the backend.
- The processed string should be returned to the client one character at a time.
- For each returned character, there should be a random pause on the server (1-5 seconds).
- All received characters should form a string in a UI textbox, updated in real-time by adding new incoming characters.
- The user cannot start another encoding process on this page while the current one is in progress.
- The user can press the “Cancel” button to cancel the currently running process.

Example of the App Functioning
- **Input:** "Hello, World!"
- **Generated String:** ' 1!1,1H1W1d1e1l3o2r1/SGVsbG8sIFdvcmxkIQ=='

What the web client receives from the server:
- Random pause... " "
- Random pause... "1"
- Random pause... "!"
- Random pause... "1"
- Etc.

What the user sees in the result text field on the web UI:
- Random pause... " "
- Random pause... " 1"
- Random pause... " 1!"
- Random pause... " 1!1"
- Etc

After completion of Part 1 please share results with us.

Part 2 – Additional requirements

- The web page should look neat; use Bootstrap or similar tooling.
- Show user a custom-made progress bar for the active processing.
- Use default IoC, package managers, and other tools to build and run the app.
- Use the latest released .NET and C# with all possible new features.
- Ensure the business logic is well-structured on both the backend and frontend, using industry-standard approaches and respective unit tests.
- Host the server-side app in a Linux Docker container.
    - Host the API and UI backend in different containers.
    - Support basic authentication using Nginx in another container.

Important details:

This should not be a simple app, this should be production ready apps with good infrastructure, good quality code, error-resistant, ready to use and ready to handle any heavy long running background process (even a few hours). Implement a good architecture and add as many features as you can.

- You should perform it as if it's the first task on the project, and you are submitting already clean, finalized code for review, without any draft code or non-working parts that are in the code but weren't completed
- You can use React or Angular, but Reat is preferable as React is a main framework
- The simulation of lengthy operations here is not without reason; you need to treat this part of the code as if it is a heavy service that operates somewhere in the backend. It's as if there is a lot of complex code there, and the architecture must be considered, including all best practices for writing such jobs. It should be resilient to failures, extensible, scalable, and you should consider that there might be a load balancer and other tools in place to ensure stable operation of heavy jobs
- It's important to consider the ability to correctly cancel the job, not just abandon everything as it is and start a new job
- And in general, you need to think everything through as much as possible. That's the essence of the task — to see if you can create a good, stable application architecture.
- You should consider that there might be more than one user, and they might misuse the application.
- Tests are also welcomed

- The more you demonstrate various best practices, patterns, and new features, the better, but they must be relevant, not just a factory for the sake of a factory that produces one object in one place in the code.

- The client usually reviews the code, evaluates it, and if everything is okay, conducts a interview based on the application code, which mainly resembles a presentation of the work—a discussion about what you did and why.

Submission Guidelines

- Code should be uploaded to Git, but there's no need to deploy to the cloud; run it locally on containers
- Provide a link to your GitHub repository or a downloadable archive. No binaries and packages please.
- Nice to have: a brief README with setup/run instructions and any assumptions made

In case of any questions, don't hesitate to ask them and clarify the requirements – contact point: **Aliaksandr Chernahrebel** (aliaksandr_chernahrebel@epam.com)